Von Compilern, Interpretern und virtuellen Maschinen

Dozent: Prof. Dr. Michael Eichberg
Kontakt: michael.eichberg@dhbw.de , Raum 149B

Version: 1.0.2.3

Kontrollfragen: kontrollfragen.de.md.html
Folien: https://delors.github.io/prog-interpreter-vms-und-compiler/folien.de.md.html
https://delors.github.io/prog-interpreter-vms-und-compiler/folien.de.md.html.pdf
Fehler melden: https://github.com/Delors/delors.github.io/issues
KI Verwendung: Bei der Erstellung der Unterlagen wurden KI Assistenten (insbesondere Claude aber ggf. auch ChatGPT, Ollama mit Gemma/LLama/Qwen oder OpenCode mit Kimi/Qwen/Deepseek...) unterstützend eingesetzt. Dies erfolgte insbesondere zur Unterstützung bei der Generierung von Grafiken (d. h. SVG Dateien), oder um sich Übersichtstabellen generieren zu lassen. Weiterhin wurde KI zur allgemeinen Qualitätssicherung eingesetzt. Inhalte, die ggf. von der KI vorgeschlagen wurden, wurden im Falle der Übernahme explizit validiert und angepasst.

1. Einführung

Interpreter

Definition

Ein Interpreter ist ein Programm, das ein Programm einer bestimmten Programmiersprache direkt ausführt.

Arbeitsweise eines Interpreters:

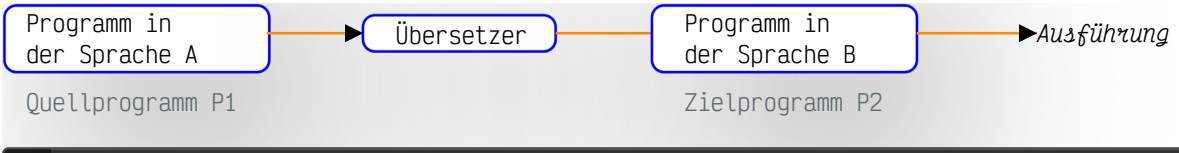
(`inpExpr` sei der aktuelle Ausdruck des auszuführenden Programms)

1. Syntaktische Analyse von `inpExpr`
2. Überführung von `inpExpr` in eine Befehlsfolge der Maschinensprache, oder der Sprache, in der das Interpreterprogramm selbst geschrieben ist (`outExpr`)
3. Ausführung von `outExpr`
4. Wiederholung der Schritte (1) bis (3) für die nächste Anweisung.

Compiler

Definition

Ein Übersetzer (Compiler) ist ein Programm, das Programme aus einer Programmiersprache A in eine Programmiersprache B übersetzt



Legende

A ist die Ausgangssprache und B die Zielsprache.

Wesentlicher Aspekt ist die semantische Korrektheit: Jedem Quellprogramm P1 in A wird genau ein Zielprogramm P2 in B zugeordnet. Das dem Quellprogramm P1 zugeordnete Zielprogramm P2 muss die gleiche Bedeutung (Semantik) wie P1 besitzen.

Mutmaßlich erfunden von Konteradmiral Grace Murray Hopper (1906–1992)

Nach eigener Aussage hat sie den Compiler aus Faulheit erfunden, und weil sie hoffte, dass „Programmierer wieder Mathematiker werden“ könnten.



Fotograf: Lynn Gilbert, 1978; Lizenz: Creative Commons Attribution-Share Alike 4.0 International

Compiler - Übersetzungsphasen

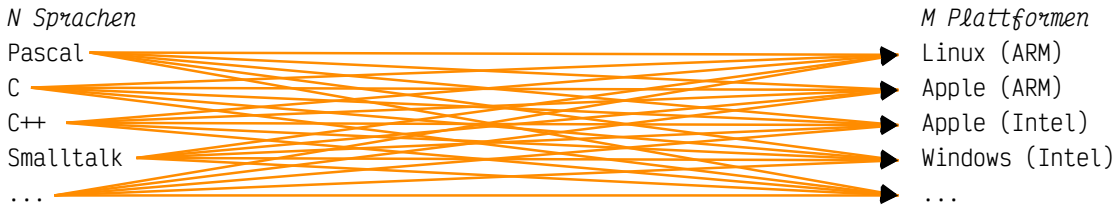
Lexikalische Analyse:
Quellprogramm wird in eine Folge von Worten zerlegt

Syntaktische Analyse:
Testet, ob das Quellprogramm den Syntaxregeln der Quellsprache entspricht. Strukturiert Worte in gültige Sätze.

Semantische Analyse:
Testet, ob alle im Quellprogramm benutzten Namen deklariert wurden und ihrem Typ entsprechend verwendet werden, usw.

Code-Generierung: Zielprogramm wird erzeugt.

Compiler - Übersetzung (Traditionell)



D. h. der Quelltext in Sprache A wird meinem spezialisierten Compiler für Sprache A und Zielplattform X in ein ausführbares Programm für X übersetzt.

★ Performance/Effizienz: Optimale Ausnutzung der jeweiligen Prozessoreigenschaften und hohe Abarbeitungsgeschwindigkeit der übersetzten Programme.

✗ Plattformabhängigkeit: Ein Programm, das in einer höheren Programmiersprache geschrieben ist, kann - bei Verfügbarkeit eines Compilers - auf jeder Maschine laufen.

Interpreter - Vor- und Nachteile

★es lassen sich relativ schnell lauffähige Programmversionen erstellen (Prototyping)

★Schnelle Änderbarkeit: geänderte Anweisungen / Deklarationen des Quellprogramms sind sofort ausführbar

★Neuübersetzung nicht notwendig

✗Längere Ausführungszeit

✗Werden Anweisungen des Quellprogramms k-mal verwendet (z.B. bei Schleifen), werden sie k-mal analysiert und überführt

✗Bei Zugriffen auf Variablen müssen die zugeordneten Adressen immer wieder bestimmt werden.

◆ Bemerkung

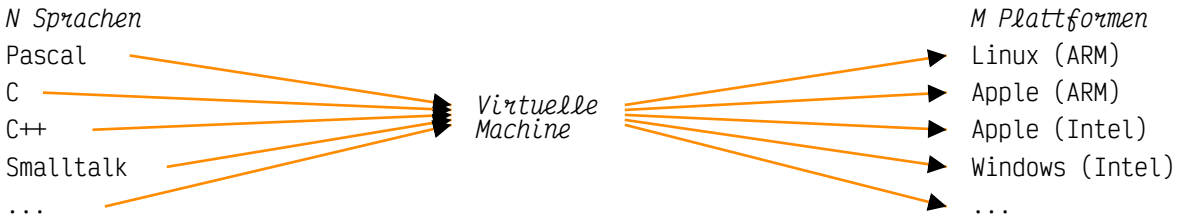
Auch heute gibt es noch *reine* Interpreter, aber Programmiersprachen wie Python und Java etc. setzen schon lange auf hybride Ansätze, die versuchen das Beste aus allen Welten vereinen.

Virtuelle Maschinen

Definition

Eine virtuelle Maschine ist ein Programm, das die Arbeit eines Prozessors in Software simuliert

- Programme einer höheren Sprache werden in eine Assembler-ähnliche Zwischensprache übersetzt.
- Der simulierte Hardware-Prozessor nutzt diese Zwischensprache und besitzt einige Software-Register
- Die Anweisungen der Zwischensprache nennt man auch Byte-Code.
- Die Zwischensprache wird von der virtuellen Maschine interpretiert.
- Eine virtuelle Maschine versteckt die spezifischen Eigenschaften eines konkreten Prozessors. Wir haben somit eine neue Abstraktionsschicht auf der Hardware-Ebene!



Eine VM verdeckt die speziellen Eigenschaften des jeweiligen Prozessortyps und dient somit als Abstraktionsschicht!

★Übersetzte Programme einer Sprache laufen auf allen Prozessortypen, für die es einen Byte-Code Interpreter(VM) gibt.

★Es wird nur ein Compiler benötigt und die Sprache wird plattformunabhängig.

Natürlich braucht man eine VM pro Prozessortyp und Platform. Aber das ist ein geringerer Aufwand als für jede Sprache einen eigenen Compiler zu schreiben.

✗Byte-Code Programme sind langsamer als Maschinenprogramme

> Just-in-time-compiler (JIT) versuchen diesen Nachteil aufzulösen. Sie Übersetzen den Byte-Code in ein Objekt-Programm für einen speziellen Prozessortyp sobald es geladen wird, oder nach einer gewissen Anzahl an Ausführungen.

Struktur eines Java-Programms

Ein Java-Programm kann aus beliebig vielen Klassen bestehen, von denen mindestens eine die `main`-Operation besitzen muss (Hauptprogrammklasse).

Aufgaben von `main`

- Objekterzeugung; d. h. der Aufbau einer anfangs minimalen Welt
- Aufruf der ersten Operation
- Sollte in der Regel keinen weitergehenden Kontrollfluss des Java-Programms enthalten

  Der Kontrollfluss wird innerhalb der Objektoperationen realisiert.
- `main` wird mit Hilfe des Java-Interpreters gestartet und ausgeführt

Java-Laufzeitumgebung

Java Interpreter:
: Programm zur Ausführung des Java-Bytecodes auf dem konkreten Rechner.

Just-In-Time-Compiler (JIT-Compiler):
: Klassen bzw. Methoden werden bei Bedarf in Code der jeweiligen Maschine übersetzt. Ggf. auf verschiedenen Optimierungsstufen.

Runtime-System:
: Stellt einem Java-Programm wichtige Ressourcen zur Verfügung.

Bytecode Verifier:
: Überprüft, ob die geladenen Bytecodes der JVM-Spezifikation entsprechen.
Klassen können über das Netz oder aus dem lokalen Dateisystem zur Laufzeit einer Java-Anwendung in das Laufzeitsystem nachgeladen werden

  Ein Teil der Sicherheitsmaßnahmen wird durch den Bytecode Verifier realisiert.

> ▲ Achtung!
>
> Die Sicherheit, die der Bytecode-Verifier und Java als solches bietet - ist nicht ausreichend, um die Sicherheit des Systems zu gewährleisten.

> ⚠ Warnung
>
> Führen Sie (Java) Programme oder auch Bibliotheken (Stichwort: *Supply-Chain Attacken*), denen Sie nicht vertrauen, niemals aus. Überzeugen Sie sich, dass die Bibliotheken, die sie verwenden legitim sind.

Java-Übersetzung

- Die Eingabe für javac sind ein oder mehrere Java-Dateien, die jeweils die eine oder mehrere Klassendefinitionen enthalten.
- Eine derartige Datei nennt man eine Übersetzungseinheit.
- Die Ausgabe ist pro Klasse X genau eine Datei X.class, die den Bytecode der Klasse enthält.

Beispiel

HelloWorld.java (im Default Package)

```

public class HelloWorld {
    public static void main(String[] args) {
        IO.println("Hello, World!");
    }
}
```

Kompilieren mit javac und Ausführen mit java

```
$ javac HelloWorld.java
$ ls
HelloWorld.class HelloWorld.java
$ java HelloWorld                                    # ACHTUNG kein ".java"!
Hello, World!
```

HelloWorld.java (im Package de.dhbw)[1]

```
package de.dhbw;

public class HelloWorld {
    public static void main(String[] args) { IO.println("Hello, World!"); }
}
```

Kompilieren mit javac und Ausführen mit java

```
$ javac de/dhbw/HelloWorld.java
$ ls de/dhbw
HelloWorld.class HelloWorld.java
$ java de.dhbw.HelloWorld                # ACHTUNG vollständiger Name der Klasse!
Hello, World!
```

[1] Zur Erinnerung: Die Datei HelloWorld.java muss im Verzeichnis de/dhbw liegen!

✎ Übung

1.1. RPN Taschenrechner kompilieren/Java Code übersetzen

Stellen Sie sicher, dass Ihr Programm für den RPN Taschenrechner durch die Klasse RPN implementiert wird und diese Klasse in der entsprechenden Java Datei gespeichert ist. Die Klasse RPN soll im Package `rpn` sein! Die Klassen für den Stack und die Liste sollen im Package `ds` liegen.

Kompilieren Sie nur Ihr Programm und die benötigten Hilfsklassen in einem Schritt mit Hilfe von `javac`. Starten Sie danach Ihr Programm mit Hilfe von *`java`*. Vergessen Sie nicht den vollqualifizierten Namen der RPN-Klasse zu verwenden. Stellen Sie auch sicher, dass Sie sich im passenden Root-Verzeichnis befinden.

Falls Sie den Code nicht haben, dann können Sie den Code von hier verwenden. Achten Sie darauf die Dateien in den entsprechenden Verzeichnissen zu speichern!

Datei: rpn/RPN.java

```java
package rpn;

import ds.Stack;

public class RPN {

    static void printAll(Stack<String> stack) {
        for (int i = 0; i < stack.size(); i++) {
            IO.print(stack.get(i) + " ");
        }
        IO.println();
    }

    public static void main(String[] args) {
        if (args.length == 0) {
            IO.println("Usage: java RPN <expr>");
            return;
        }
        // Main logic
        Stack<String> infix = new Stack<>();
        Stack<Double> ops = new Stack<>();
        for (String arg : args) {
            switch (arg) {
                case "+":
                    ops.push(ops.pop() + ops.pop());
                    infix.push("(" + infix.pop() + " + " + infix.pop() + ")");
                    break;
                case "*":
                    ops.push(ops.pop() * ops.pop());
                    infix.push("(" + infix.pop() + " * " + infix.pop() + ")");
                    break;
                default:
                    infix.push(arg);
                    ops.push(Double.parseDouble(arg));
            }
        }
        IO.println(infix.peek() + " = " + ops.peek());

```

```
        printAll(infix);
    }
}
```

Datei: ds/Stack.java

```
package ds;

import java.util.NoSuchElementException;

public class Stack<T> extends List<T> {

    public void push(T element) {
        add(element);
    }

    public T pop() {
        if (size() == 0) {
            throw new NoSuchElementException();
        }
        T element = get(size() - 1);
        remove(size() - 1);
        return element;
    }

    public T peek() {
        if (size() == 0) {
            throw new NoSuchElementException();
        }
        return get(size() - 1);
    }
}
```

Datei: ds/List.java

```
package ds;

import static java.lang.System.arraycopy;

public class List<T> {

    private T[] elements;
    private int count;

    public List(int size) {
        @SuppressWarnings("unchecked")
        var tElements = (T[]) new Object[Math.max(size, 16)];
        elements = tElements;
        count = 0;
    }

    public List() {
        this(16);
    }

    public int size() {
        return count;
    }

```

```
    public T get(int index) {
        if (index < 0 || index >= count) {
            throw new IndexOutOfBoundsException();
        }
        return elements[index];
    }

    public void add(T element) {
        if (count == elements.length) {
            Object[] newElements = new Object[Math.min(elements.length * 2, 1000)];
            System.arraycopy(elements, 0, newElements, 0, elements.length);
            @SuppressWarnings("unchecked")
            var tElements = (T[]) newElements;
            elements = tElements;
        }
        elements[count++] = element;
    }

    @SafeVarargs
    final public void addAll(T... elements) {
        for (T element : elements) {
            add(element);
        }
    }

    public void set(int index, T element) {
        if (index < 0 || index >= count) {
            throw new IndexOutOfBoundsException();
        }
        elements[index] = element;
    }

    public String toString() {
        StringBuilder sb = new StringBuilder();
        sb.append("[");
        for (int i = 0; i < count; i++) {
            if (i > 0) {
                sb.append(", ");
            }
            sb.append(elements[i]);
        }
        sb.append("]");
        return sb.toString();
    }

    public void remove(int index) {
        if (index < 0 || index >= count) {
            throw new IndexOutOfBoundsException();
        }

        if (count < elements.length / 4 && elements.length > 16) {
            @SuppressWarnings("unchecked")
            T[] newElements = (T[]) new Object[Math.max(elements.length / 2, 16)];
            arraycopy(elements, 0, newElements, 0, index);
            arraycopy(elements, index + 1, newElements, index, count - index - 1);
            elements = newElements;
            count--;
        } else {
```

```
83              arraycopy(elements, index + 1, elements, index, count - index - 1);
84              count--;
85          }
86      }
87
88  }
```